Scattering of Relativistic Dirac Electrons: From Theory to Numerical Models

Zhengjiang Li

April 24, 2025

Contents

1 Review of Basic Knowledge **1**

2 Perturbation Theory in Scattering of Dirac Electrons **2**

3 Numerical Result **5**

4 Summary and Outlook **8**

References **8**

Appendix **8**

1 Review of Basic Knowledge

In quantum mechanics, we consider scattering in the aspect of a single particle wave function obeying the Schrödinger wave equation:

$$\hat{H}\psi(\boldsymbol{r},t) = i\hbar\partial_t\psi(\boldsymbol{r},t) \tag{1.1}$$

$$\hat{H} = \hat{H}_0 + \hat{V} = \frac{\hat{\boldsymbol{P}}^2}{2m} + \hat{V} \tag{1.2}$$

One can get the time independt Schrödinger equation via seperation of variables:

$$\hat{H}\psi(\boldsymbol{r}) = E\psi(\boldsymbol{r}) \tag{1.3}$$

In Dirac theory, the Hamiltonian is:

$$\hat{H} = \hat{\alpha}_0 mc^2 + \sum_{j=1}^{3} \hat{\alpha}_j \hat{p}_j c, \quad \text{where } \hat{p}_j \equiv -i\hbar\partial_j, \tag{1.4}$$

where in the Dirac represenation,

$$\hat{\alpha}_0 = \begin{bmatrix} \hat{I} & \hat{0} \\ \hat{0} & -\hat{I} \end{bmatrix}, \quad \hat{\alpha}_1 = \begin{bmatrix} \hat{0} & \hat{\sigma}_1 \\ \hat{\sigma}_1 & \hat{0} \end{bmatrix}, \tag{1.5}$$

$$\hat{\alpha}_2 = \begin{bmatrix} \hat{0} & \hat{\sigma}_2 \\ \hat{\sigma}_2 & \hat{0} \end{bmatrix}, \quad \hat{\alpha}_3 = \begin{bmatrix} \hat{0} & \hat{\sigma}_3 \\ \hat{\sigma}_3 & \hat{0} \end{bmatrix}, \tag{1.6}$$

and we expect it still obey equation (1.1) with Hamiltonian has the form of (1.4).

The gamma matrix γ^μ in the dirac represenation is defined as: $\gamma^0 = \hat{\alpha}_0,\ \gamma^i = \gamma^0\hat{\alpha}_i$.

The Dirac equation can be formulated in the following way:

$$(i\gamma^\mu\partial_\mu - m)\psi(\boldsymbol{r}, t) = 0 \tag{1.7}$$

$$\gamma^\mu p_\mu - m = 0 \tag{1.8}$$

The plane wave solution of Dirac equation can be naturally derived as (for positive energy as an example):

$$\psi^s(\boldsymbol{r}, t) = u^s(p)\exp(-ip^\mu x_\mu) \tag{1.9}$$

with a new degree of freedom spin s.

2 Perturbation Theory in Scattering of Dirac Electrons

In the relativistic quantum mechanics level, we regard $\psi(\boldsymbol{r})$ as a single particle wave function, and a scattering process can be studied under this formalism. In this chapter we will show how to use the (old-fashioned) perturbation theory and Born approximation in the scattering of Dirac electrons, and give an example of numerical calculation.

Given a Hamiltonian, the Lippmann–Schwinger equation is given as following:

$$|\psi\rangle = |\phi\rangle + \frac{1}{E - H_0}V|\psi\rangle, \tag{2.1}$$

The L-S equation is equivalent to the Schrödinger equation, and we have obtained this in class via the Green function approach:

$$|\psi_s\rangle = G_0 V|\psi\rangle, \tag{2.2}$$

$$|\psi\rangle = |\psi_i\rangle + G_0 V|\psi\rangle \tag{2.3}$$

where

$$|\psi\rangle = |\psi_i\rangle + |\psi_s\rangle, \tag{2.4}$$

$$G_0 = \frac{1}{E - H_0}. \tag{2.5}$$

Given a Dirac Hamiltonian (as the H_0 in the above formula), the Green function (Lippmann–Schwinger kernel) can be expended in the momentum eigenstates:

$$G_0(E) = \sum_s \left[\frac{|k, s, +\rangle\langle k, s, +|}{E - E_k \pm i\epsilon} + \frac{|k, s, -\rangle\langle k, s, -|}{E + E_k \pm i\epsilon}\right] \tag{2.6}$$

Here $|k, s, +\rangle$ and $|k, s, -\rangle$ indicate the momentum eigenstates of the positive and negative energy.

By iteration, one can finally calculate the wave function as following:

$$\psi(\mathbf{r}) = \psi_i(\mathbf{r}) + \int d^d r' \, \langle \mathbf{r}|\hat{G}_0|\mathbf{r}'\rangle \, V(\mathbf{r}') \, \psi_i(\mathbf{r}') + \int d^d r' \, d^d r'' \, \langle \mathbf{r}|\hat{G}_0|\mathbf{r}'\rangle \, V(\mathbf{r}') \, \langle \mathbf{r}'|\hat{G}_0|\mathbf{r}''\rangle \, V(\mathbf{r}'') \, \psi_i(\mathbf{r}'') + \cdots \tag{2.7}$$

and,

$$\psi_s(\mathbf{r}) = \int d^3 r' \, \langle \mathbf{r}|\hat{G}_0|\mathbf{r}'\rangle \, V(\mathbf{r}') \, \langle \mathbf{r}'|(\hat{I} + \hat{G}\hat{V})|\psi_i\rangle. \tag{2.8}$$

If we want to do these calculations based on the perturbation theory, the keypoint is to get the matrix elements $\langle \boldsymbol{r}|G_0|\boldsymbol{r}'\rangle$ of G_0:

$$\begin{aligned}
\langle \boldsymbol{r}|G_0|\boldsymbol{r}'\rangle &= \langle \boldsymbol{r}|G_0 \sum_s \int \frac{d^3k}{(2\pi)^3 2E_k} \left(|k,s,+\rangle\langle k,s,+| + |k,s,-\rangle\langle k,s,-|\right) |\boldsymbol{r}'\rangle \\
&= \langle \boldsymbol{r}| \sum_s \int \frac{d^3k}{(2\pi)^3 2E_k} \left(\frac{|k,s,+\rangle\langle k,s,+|}{E - E_k \pm i\epsilon} + \frac{|k,s,-\rangle\langle k,s,-|}{E + E_k \pm i\epsilon} \right) |\boldsymbol{r}'\rangle \\
&= \sum_s \int \frac{d^3k}{(2\pi)^3 2E_k} \left(\frac{\langle \boldsymbol{r}|k,s,+\rangle\langle k,s,+|\boldsymbol{r}'\rangle}{E - E_k \pm i\epsilon} + \frac{\langle \boldsymbol{r}|k,s,-\rangle\langle k,s,-|\boldsymbol{r}'\rangle}{E + E_k \pm i\epsilon} \right) \\
&= \sum_s \int \frac{d^3k}{(2\pi)^3 2E_k} \left(\frac{e^{i\mathbf{k}\cdot(\mathbf{r}-\mathbf{r}')} u^s_{\mathbf{k}} u^{s\dagger}_{\mathbf{k}}}{E - E_k \pm i\epsilon} + \frac{e^{-i\mathbf{k}\cdot(\mathbf{r}-\mathbf{r}')} v^s_{-\mathbf{k}} v^{s\dagger}_{-\mathbf{k}}}{E + E_k \pm i\epsilon} \right) \\
&= \int \frac{d^3k}{(2\pi)^3 2E_k} \left(\frac{e^{i\mathbf{k}\cdot(\mathbf{r}-\mathbf{r}')} (\not{k} + m)\gamma^0}{E - E_k \pm i\epsilon} + \frac{e^{-i\mathbf{k}\cdot(\mathbf{r}-\mathbf{r}')} (\not{k} - m)\gamma^0}{E + E_k \pm i\epsilon} \right)
\end{aligned} \tag{2.9}$$

Here we use the single-particle wavefunctions as following. The base vectors are matrix themselves, and we choose the normalization

$$\langle \mathbf{r}|\mathbf{k},s,+\rangle = e^{i\mathbf{k}\cdot\mathbf{r}} \, u^s_{\mathbf{k}}, \qquad \langle \mathbf{r}|\mathbf{k},s,-\rangle = e^{-i\mathbf{k}\cdot\mathbf{r}} \, v^s_{-\mathbf{k}} \tag{2.10}$$

We also use the normalization and summing relation of dirac spinors[1, 2]:

$$u^{r\dagger}(k) \, u^s(k) = 2E_{\mathbf{k}} \, \delta^{rs}. \tag{2.11}$$

$$\sum_s u^s_{\mathbf{k}} u^{s\,\dagger}_{\mathbf{k}} = (\not{k} + m)\gamma^0, \quad \sum_s v^s_{-\mathbf{k}} v^{s\,\dagger}_{-\mathbf{k}} = (\not{k} - m)\gamma^0 \tag{2.12}$$

where $\not{k} = \gamma^\mu k_\mu$. To get the result, we first let $\mathbf{r} - \mathbf{r}' = \mathbf{R}$ along the z axis, and the integral(for example, the positive energy part) becomes:

$$\begin{aligned}
&\int \frac{dk d\theta d\phi}{(2\pi)^3 2E_k} k^2 \sin\theta \left(\frac{e^{ikR\cos\theta} (\gamma^0 E_k + k\left(\gamma^1 \sin\theta\cos\phi + \gamma^2 \sin\theta\sin\phi + \gamma^3\cos\theta\right) + m)\gamma^0}{E - E_k \pm i\epsilon} \right) \\
&= \int_0^\infty \frac{dk}{(2\pi)^2 2E_k} \int_0^\pi d\theta \; k^2 \sin\theta \left(\frac{e^{ikR\cos\theta} (\gamma^0 E_k + k\gamma^3\cos\theta + m)\gamma^0}{E - E_k \pm i\epsilon} \right)
\end{aligned} \tag{2.13}$$

Here we first integral over ϕ. Notice that the first two terms with $\cos\phi, \sin\phi$ will be zero. Then considering the integration of θ, notice that:

$$\int_{-1}^{1} d\cos\theta e^{ikR\cos\theta} = \frac{e^{ikR} - e^{-ikR}}{ikR} \tag{2.14}$$

$$\int_{-1}^{1} d\cos\theta e^{ikR\cos\theta}\cos\theta = \frac{\partial}{iR\partial k}\int_{-1}^{1} d\cos\theta e^{ikR\cos\theta} = \frac{\partial}{iR\partial k}\frac{e^{ikR}-e^{-ikR}}{ikR}$$
$$= \frac{e^{ikR}(1-ikR)-e^{-ikR}(1+ikR)}{k^3R^2} \tag{2.15}$$

So (2.13) can be reduce to:

$$\int_0^\infty \frac{dk}{(2\pi)^2 2E_k}\, k^2 \left(\frac{(\gamma^0 E_k + m)\gamma^0}{E-E_k\pm i\epsilon}\frac{e^{ikR}-e^{-ikR}}{ikR} - \frac{k\gamma^3\gamma^0}{E-E_k\pm i\epsilon}\frac{e^{ikR}(1-ikR)-e^{-ikR}(1+ikR)}{k^2R^2}\right)$$
$$= \int_{-\infty}^\infty \frac{dk'}{(2\pi)^2 2E_k}\left(\frac{(\gamma^0 E_k + m)\gamma^0}{E-E_k\pm i\epsilon}\frac{k'e^{ik'R}}{iR} - \frac{\gamma^3\gamma^0 k'}{E-E_k\pm i\epsilon}\frac{e^{ik'R}(1-ik'R)}{R^2}\right) \tag{2.16}$$

using the relation: $E=\sqrt{k^2+m^2},\ E_k=\sqrt{k'^2+m^2}$, there will be poles at k'=k. The denominator can be express as follow:

$$\frac{1}{\sqrt{k^2+m^2}-\sqrt{k'^2+m^2}} = \frac{\sqrt{k^2+m^2}+\sqrt{k'^2+m^2}}{k^2-k'^2} \tag{2.17}$$

By applying the similar ϵ trick, this can be avoided, and the result can be obtained by residue theorem:

$$\langle \boldsymbol{r}|G_0|\boldsymbol{r}'\rangle = \frac{i}{(2\pi)}\left(\frac{(\gamma^0 E+m)\gamma^0 k e^{ikR}}{2ikR} - \frac{\gamma^3\gamma^0 k e^{ikR}(1-ikR)}{2kR^2}\right)$$
$$= \left(\frac{(E+m\gamma^0)e^{ikR}}{4\pi R} - \frac{\gamma^3\gamma^0 e^{ikR}(i+kR)}{4\pi R^2}\right) \tag{2.18}$$

We can do the similar operation for the negative part, the only difference is that when $E>0$, there is no poles here, so the residue is zero, and this term does not contribute. We can conclude that the first term in (2.9) only contributes when we have positive energy, while the second term only conreibute when the energy is negative.

Now consider the scattering amplitude. We have the wave function:

$$\psi_s(\mathbf{r}) = \int d^3r'\, \langle \mathbf{r}|\hat{G}_0|\mathbf{r}'\rangle\, V(\mathbf{r}')\, \langle \mathbf{r}'|(\hat{I}+\hat{G}\hat{V})|\psi_i\rangle. \tag{2.19}$$

when $r\to\infty$, $\langle \boldsymbol{r}|G_0|\boldsymbol{r}'\rangle \approx \frac{(E+m\gamma^0-\gamma^3\gamma^0 k)e^{ik(r-\hat{\mathbf{r}}\cdot\mathbf{r}')}}{4\pi r}$. The scattered wave function is:

$$\psi_s(\mathbf{r}) \xrightarrow{r\to\infty} \int d^3r'\, \frac{(E+m\gamma^0-\gamma^3\gamma^0 k)e^{ik(r-\hat{\mathbf{r}}\cdot\mathbf{r}')}}{4\pi r}\, V(\mathbf{r}')\langle \mathbf{r}'|(\hat{I}+\hat{G}\hat{V})|\psi_i\rangle \tag{2.20}$$

So the scattering amplitude is:

$$f = \int d^3r'\, \frac{(E+m\gamma^0-\gamma^3\gamma^0 k)e^{-ik\hat{\mathbf{r}}\cdot\mathbf{r}'}}{4\pi}\, V(\mathbf{r}')\, \langle \mathbf{r}'|(\hat{I}+\hat{G}\hat{V})|\psi_i\rangle$$
$$= \int d^3r'\, \frac{(E+m\gamma^0-\gamma^3\gamma^0 k)}{4\pi\cdot 2E}\, \langle \mathbf{r}'|u_k^{s\dagger}u_k^s e^{-ik\hat{\mathbf{r}}\cdot\mathbf{r}'}(\hat{V}+\hat{V}\hat{G}\hat{V})|\psi_i\rangle$$
$$= \frac{(E+m\gamma^0-\gamma^3\gamma^0 k)}{4\pi\cdot 2E}\, \langle \mathbf{k_f}, s|u_k^s(\hat{V}+\hat{V}\hat{G}\hat{V})|\mathbf{k}_f, s\rangle \tag{2.21}$$

One can consider the first two order $V+VG_0V$:

$$f = f_1 + f_2 \tag{2.22}$$

$$
\begin{aligned}
f_1 &= \int d^3r' \, \frac{(E + m\gamma^0 - \gamma^3\gamma^0 k)e^{-ik\hat{\mathbf{r}}\cdot\mathbf{r}'}}{4\pi} \, V(\mathbf{r}') \, \langle \mathbf{r}' | k_i, s \rangle \\
&= \int d^3r' \, \frac{(E + m\gamma^0 - \gamma^3\gamma^0 k)e^{i(k_i - k_f)\cdot\mathbf{r}'}}{4\pi} \, V(\mathbf{r}') \, u^s_{k_i} \qquad (2.23)
\end{aligned}
$$

$$
\begin{aligned}
f_2 &= \int d^3r' d^3r'' \, \frac{(E + m\gamma^0 - \gamma^3\gamma^0 k)e^{-ik\hat{\mathbf{r}}\cdot\mathbf{r}'}}{4\pi} \, V(\mathbf{r}') \, \langle \mathbf{r}' | \hat{G}_0 \hat{V} | \psi_i \rangle \\
&= \int d^3r' d^3r'' \, \frac{(E + m\gamma^0 - \gamma^3\gamma^0 k)e^{-ik_f\cdot\mathbf{r}'}}{4\pi} \, V(\mathbf{r}') \, \langle \mathbf{r}' | \hat{G}_0 | \mathbf{r}'' \rangle V(r'') \langle \mathbf{r}'' | k_i, s \rangle \\
&= \int d^3r' d^3r'' \, \frac{(E + m\gamma^0 - \gamma^3\gamma^0 k)e^{i(k_i\cdot\mathbf{r}'' - k_f\cdot\mathbf{r}')}}{4\pi} \, V(\mathbf{r}') \, \frac{(E + m\gamma^0 - \gamma^3\gamma^0 k - i\gamma^3\gamma^0/R)e^{ikR}}{4\pi R} V(\mathbf{r}'') u^s_{k_i}
\end{aligned} \qquad (2.24)
$$

We can define the following quantity as the probability of the scattering which maintain the Lorentz invariance:

$$
|f|^2 = \bar{f} f = f^\dagger \gamma^0 f \qquad (2.25)
$$

3 Numerical Result

Uniform spherical potential

In this section, I try the numerical calculation of the scattering amplitude for the first and second born approximation based on the calculation in the above section. For easier calculation, here we shift to weyl represenation, with a slightly different γ^0. We choose the uniform spherical potential, and always let the electron incident along z axis:

$$
V(\vec{r}) = \begin{cases} -U & \text{for } |\vec{r}| < R \\ 0 & \text{for } |\vec{r}| \geq R \end{cases} \qquad (3.1)
$$

First we check the energy spectrum with fixed outgoing angle. We choose U=-0.3 and R=1. We change the emergent beams along x-axis or z-axis, and Generally, the result is shown in Fig.1 lives up to expectations.

Then, we calculate the amplitude with different emergent angles. With larger θ(backwards), the amplitude decrease, which is accordance with expectations. See Fig.2.

Mimic Stern-Gerlach experiment

To further explore the difference from the spin degree of freedom, we apply the magnetic field along the x axis, while keep electron moving along z axis. The potential is given by[3], with an expension to the Dirac electrons:

$$
\hat{V}(r) = -U \cdot x \cdot \Sigma_x \qquad (3.2)
$$

$$
\Sigma_x = \mathbf{1} \otimes \sigma_x \qquad (3.3)
$$

Then, we plot the amplitute with different θ and ϕ. For fixed ϕ=0, one can observe a deflection angle of about 50°. When we fix the angle at about this number, and change different ϕ, the amplitude reach the maxium at both $\phi = 0$ and $\phi = \pi$, which means that the electrons incident along z axis will be seperate into

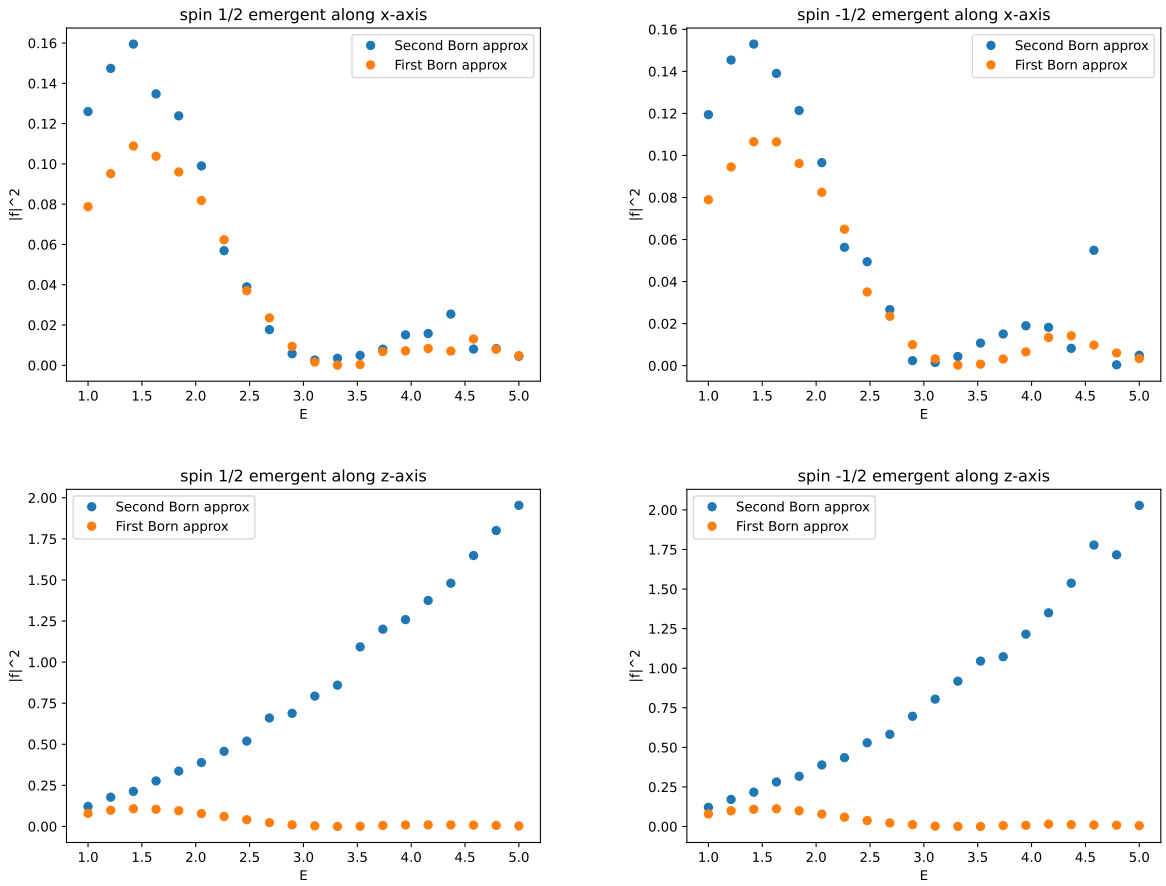


Figure 1: Energy spectrum

two beams along the x axis, and clearly reproduce the result of the Stern-Gerlach experiment. Notice that here we choose the spin along z-axis, so on x axis, there will be two components exist simultaneously.

Finally, we can compare with the result of the spherical potential with no spin interaction in the last figure in Fig.3, and there is no splitting at all, which confirm our result is really due to the spin interaction.

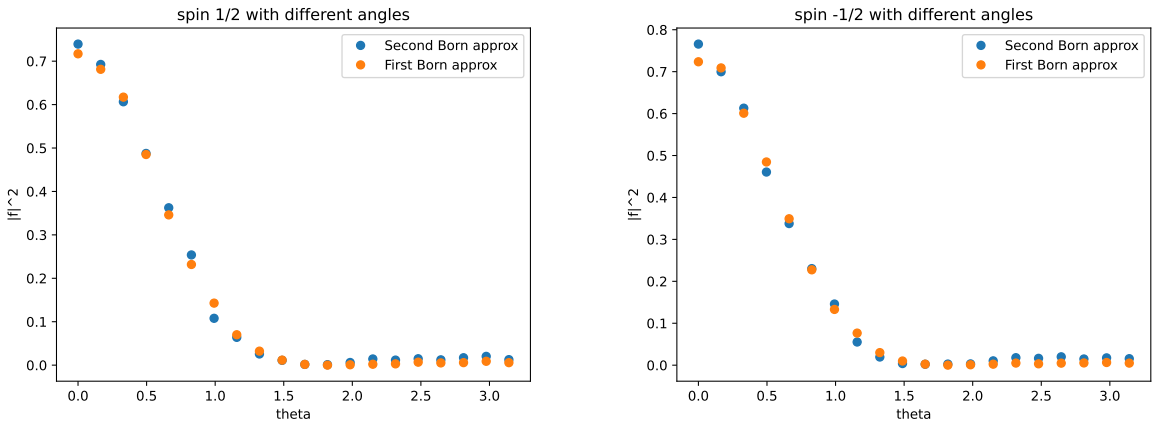


Figure 2: Scattering with theta

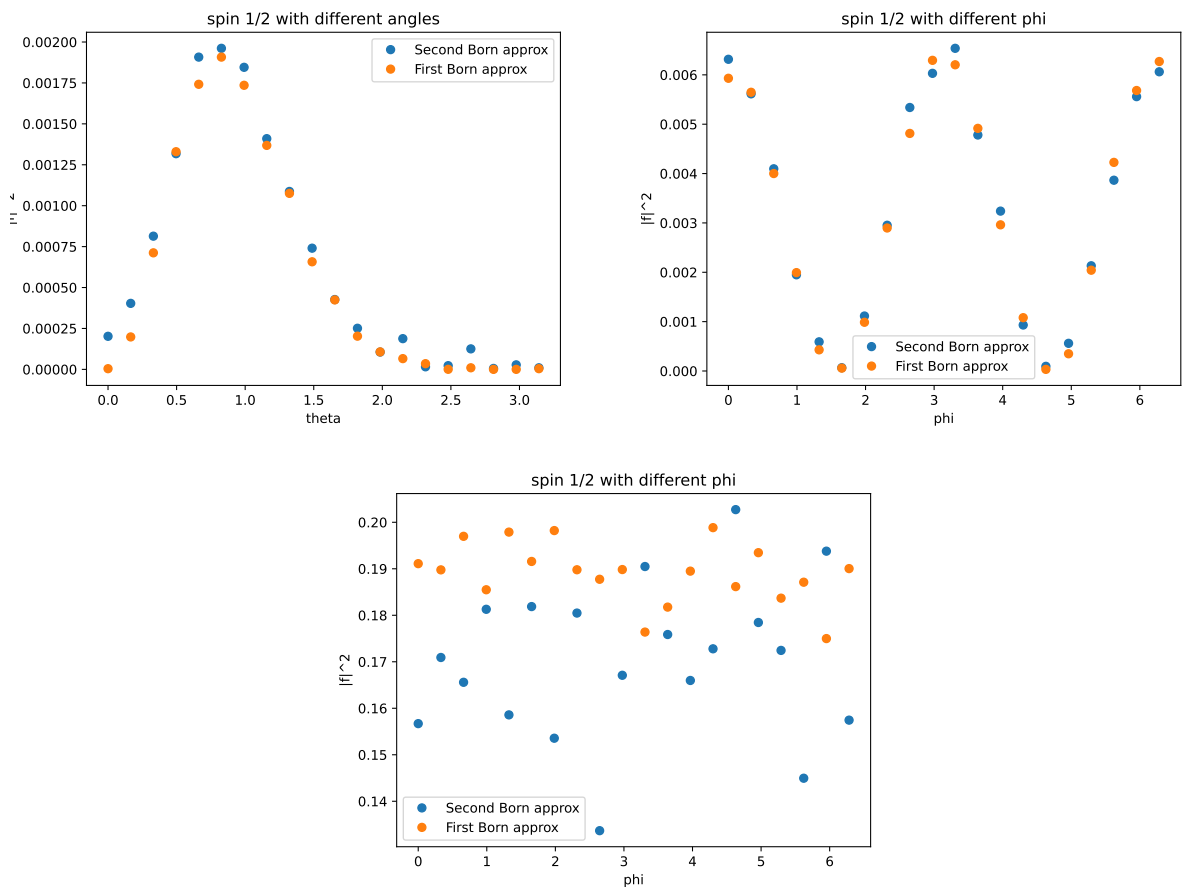


Figure 3: scattering with different θ and ϕ

4 Summary and Outlook

In this report, I derive the theories of the scattering of Dirac electrons in the relativistic quantum mechanics, and applying different potentials for simulation. By adding the potential related to spin, we show the deflection of electrons. which is a phenomenon that will not be included in non-relativistic quantum mechanics. However, as we are choosing the Weyl represenation in the numerical simulation, the spinor of injecting state is not exactly the eigenstates of the spin operators, which makes the seperation of spin 1/2 and spin -1/2 difficult. In the future it will be interesting to further refine this problem with different represenations.

The old fashioned perturbation theory is also useful in quantum field theory when considering the scattering of particles, since it will provide clear physical processes where the intermediate particle is on-shell. However due to the limitation of space, we will not discuss it in this report.

References

[1] Mark D. Schwartz. *Quantum Field Theory and the Standard Model.* Cambridge University Press, Cambridge, 2013.

[2] Michael E. Peskin and Daniel V. Schroeder. *An Introduction to quantum field theory.* Addison-Wesley, Reading, USA, 1995.

[3] KM Fonseca-Romero. A quantum treatment of the stern-gerlach experiment. *arXiv preprint arXiv:2402.14930*, 2024.

Appendix

Scattering of Dirac Electron_Zhengjiang

April 24, 2025

1 Scattering of Dirac Electron

Here we calculate the scattering amplitude via Monte Carlo integration, following the method we use in the homework.

1.1 Basic Quantities

[1]:
```
import numpy as np

# define 2x2 unitary matrix and Pauli matrix
I2 = np.identity(2, dtype=complex)
zero2 = np.zeros((2, 2), dtype=complex)

sigma_x = np.array([[0, 1], [1, 0]], dtype=complex)
sigma_y = np.array([[0, -1j], [1j, 0]], dtype=complex)
sigma_z = np.array([[1, 0], [0, -1]], dtype=complex)

#  gamma matrix Dirac
I4=np.identity(4, dtype=complex)

gamma_0 = np.block([
    [zero2,  I2],
    [I2, zero2]
])

gamma_1 = np.block([
    [zero2,  sigma_x],
    [-sigma_x, zero2]
])

gamma_2 = np.block([
    [zero2,  sigma_y],
    [-sigma_y, zero2]
])

gamma_3 = np.block([
    [zero2,  sigma_z],
    [-sigma_z, zero2]
```

```
])

gamma_5 = 1j * gamma_0 @ gamma_1 @ gamma_2 @ gamma_3


gamma_list = [gamma_0, gamma_1, gamma_2, gamma_3, gamma_5]
gamma_labels = ['gamma^0', 'gamma^1', 'gamma^2', 'gamma^3', 'gamma^5']

# output
for label, gamma in zip(gamma_labels, gamma_list):
    print(f"{label} =\n{gamma}\n")
```

```
gamma^0 =
[[0.+0.j 0.+0.j 1.+0.j 0.+0.j]
 [0.+0.j 0.+0.j 0.+0.j 1.+0.j]
 [1.+0.j 0.+0.j 0.+0.j 0.+0.j]
 [0.+0.j 1.+0.j 0.+0.j 0.+0.j]]

gamma^1 =
[[ 0.+0.j  0.+0.j  0.+0.j  1.+0.j]
 [ 0.+0.j  0.+0.j  1.+0.j  0.+0.j]
 [-0.-0.j -1.-0.j  0.+0.j  0.+0.j]
 [-1.-0.j -0.-0.j  0.+0.j  0.+0.j]]

gamma^2 =
[[ 0.+0.j  0.+0.j  0.+0.j -0.-1.j]
 [ 0.+0.j  0.+0.j  0.+1.j  0.+0.j]
 [-0.-0.j  0.+1.j  0.+0.j  0.+0.j]
 [-0.-1.j -0.-0.j  0.+0.j  0.+0.j]]

gamma^3 =
[[ 0.+0.j  0.+0.j  1.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j -1.+0.j]
 [-1.-0.j -0.-0.j  0.+0.j  0.+0.j]
 [-0.-0.j  1.-0.j  0.+0.j  0.+0.j]]

gamma^5 =
[[-1.+0.j  0.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j -1.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  1.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j  1.+0.j]]
```

1.2 First Born Approximation

First we calculate the first born approximation

$$f_1 = \int d^3r' \, \frac{(E + m\gamma^0 - \gamma^3\gamma^0 k)e^{i(k_i - k_f)\cdot \mathbf{r}'}}{4\pi} \, V(\mathbf{r}') \, u^s_{k_i}$$

An expedient way to calculate these integrals is Monte Carlo integration. Suppose we want to compute an integral of the form

$$I = \int_V d^3r\, F(\mathbf{r}),$$

taken over some domain of volume V. We randomly sample N points, $\{F_1, F_2, \dots, F_N\}$. Then the estimate for the integral is

$$I \approx \frac{V}{N} \sum_{n=1}^{N} F_n.$$

We will assume computational units, $m = \hbar = 1$. First, we import the Scipy libraries, and the Matplotlib library for plotting. Then, we define a function that calculates the first term of the Born series via Monte Carlo integration:

[3]:
```
#Calculation of Dirac spinor under Weyl representation

import numpy as np

def sigma_mu():
    I = np.identity(2, dtype=complex)
    sigma_1 = np.array([[0, 1], [1, 0]], dtype=complex)
    sigma_2 = np.array([[0, -1j], [1j, 0]], dtype=complex)
    sigma_3 = np.array([[1, 0], [0, -1]], dtype=complex)

    sigma = [I, sigma_1, sigma_2, sigma_3]
    barsigma = [I, -sigma_1, -sigma_2, -sigma_3]
    return sigma, barsigma

def p_dot_sigma(p, sigma_list):
    return sum(p[mu] * sigma_list[mu] for mu in range(4))

def matrix_sqrt(mat):
    """
    Compute the matrix square root using spectral decomposition.
    Only defined for Hermitian positive-definite 2x2 matrices.
    """
    vals, vecs = np.linalg.eigh(mat)  # eigen decomposition: A = V D V^†
    sqrt_vals = np.sqrt(np.maximum(vals, 0))  # ensure numerical stability
    sqrt_mat = vecs @ np.diag(sqrt_vals) @ np.linalg.inv(vecs)
    return sqrt_mat

def u_spinor_weyl(p, xi):
    """
    Construct u(p) in Weyl representation using matrix square root.
    Parameters:
```

```
        p : (E, px, py, pz)
        xi : 2-component spinor
    Returns:
        u(p): 4-component Dirac spinor
    """
    sigma, barsigma = sigma_mu()

    p_sigma = p_dot_sigma(p, sigma)
    p_barsigma = p_dot_sigma(p, barsigma)

    sqrt_p_sigma = matrix_sqrt(p_sigma)
    sqrt_p_barsigma = matrix_sqrt(p_barsigma)

    upper = sqrt_p_barsigma @ xi
    lower = sqrt_p_sigma @ xi

    u = np.vstack((upper, lower))
    return u

# 4-momentum E, px, py, pz
p = np.array([2.0, 0.0, 0.0, np.sqrt(3)])  # m = 1   E = 2

# spin-up
xi = np.array([[1.0], [0.0]], dtype=complex)

u = u_spinor_weyl(p, xi)
print("u(p) =\n", u)
```

```
u(p) =
 [[0.51763809+0.j]
 [0.        +0.j]
 [1.93185165+0.j]
 [0.        +0.j]]
```

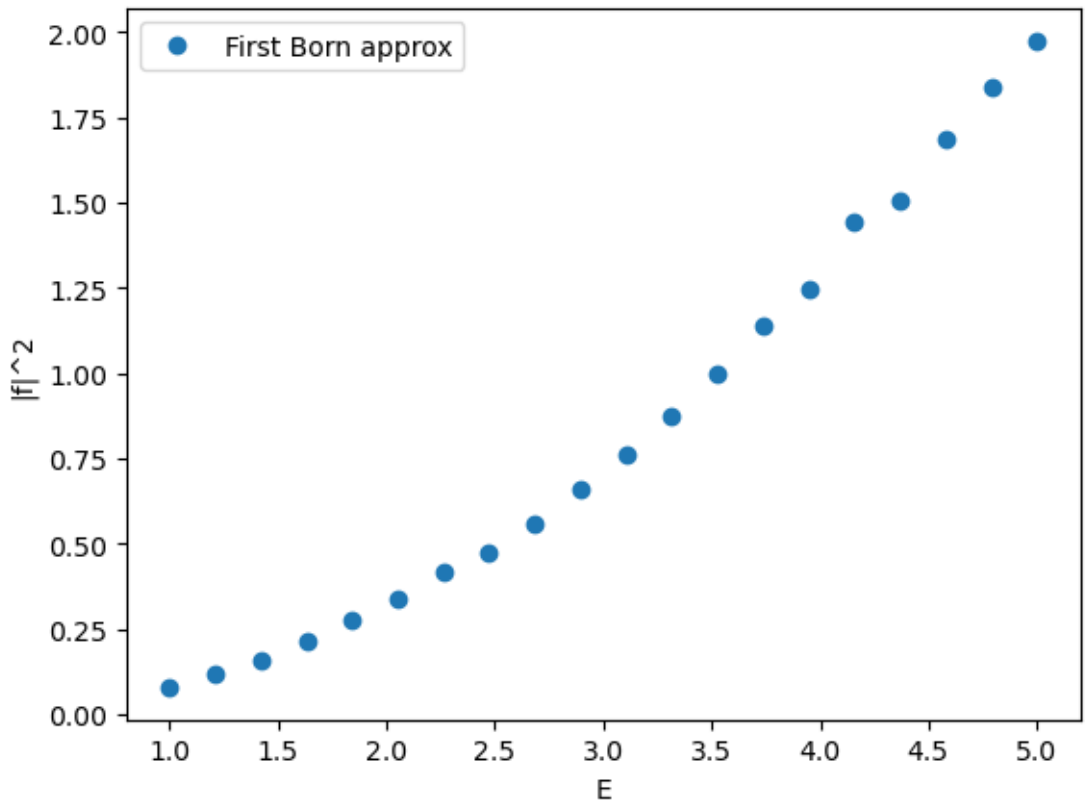
[5]:
```
%matplotlib inline
import numpy as np
import scipy.linalg
import matplotlib.pyplot as plt

## Function to estimate the first Born contribution to the scattering amplitude␣
 ↪f.
## The inputs are:
##
## Vfun   -- A function object.  The function should take an array [x, y, z]
##           and (optionally) other parameters. It should return V(x,y,z).
## ki, kf -- Initial and final wavevectors, which should have equal magnitudes.
## L      -- Size of sampling box.  Each coordinate is drawn from [-L, L].
```

```
## N       -- Number of Monte Carlo runs; default is 5000.
## Vargs   -- A tuple of additional parameters to pass Vfun.

def fborn1(Vfun, ki, kf, L, N, xi, m=1, Vargs=()):
    volume = (2*L)**3     # Spatial volume of integration
    f1 = np.zeros((4, 1), dtype=complex)            # Scattering amplitude
    k=np.sqrt(np.dot(ki,ki))
    for n in range(N):
        r1  = 2 * L * np.random.random(3) - L # Each coordinate is -L to L
        dkr = np.dot(ki - kf, r1)
        E=np.sqrt(np.dot(kf, kf)+m**2)
        p=np.concatenate(([E], ki))
        ui=u_spinor_weyl(p, xi)
        f1 +=  Vfun(r1, *Vargs) * np.exp(1j * dkr)*(E*I4+m*gamma_0-k*gamma_3 @␣
 ↪gamma_0) @ ui / (4 * np.pi)
    return volume * f1 / N
```

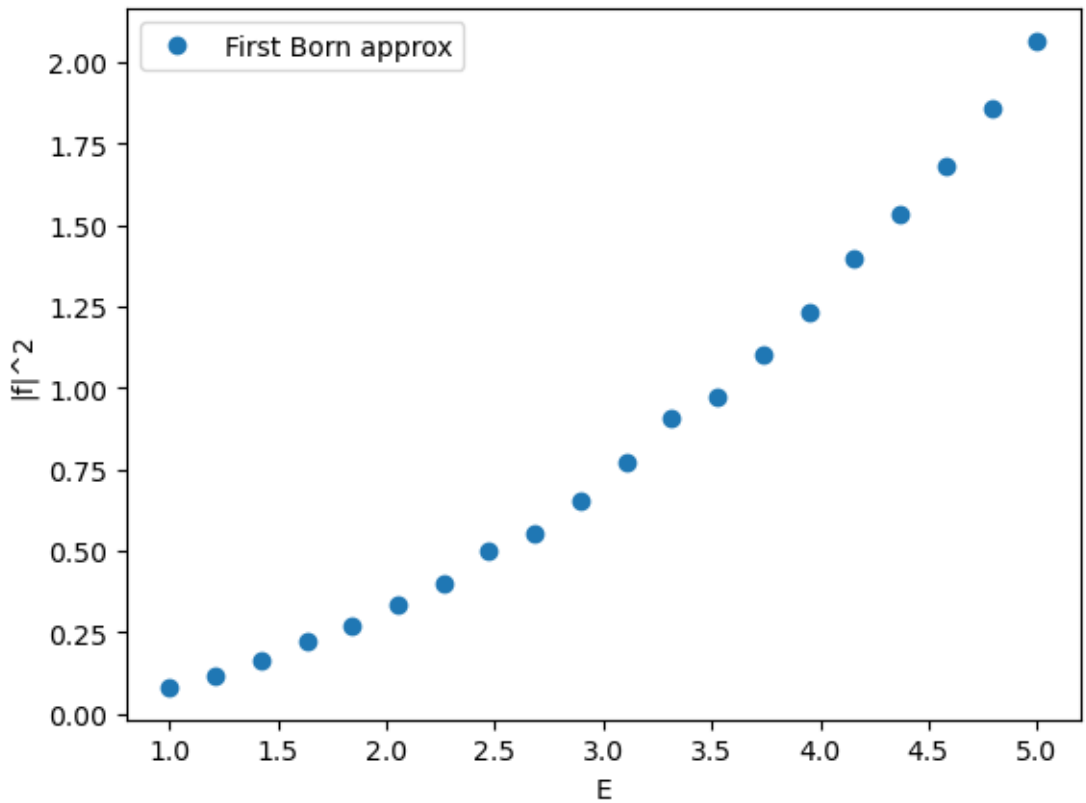
$$f_1 = \int d^3r' \, \frac{(E + m\gamma^0 - \gamma^3\gamma^0 k)e^{i(k_i - k_f)\cdot \mathbf{r}'}}{4\pi} \, V(\mathbf{r}') \, u^s_{k_i}$$

1.2.1 Energy Spectrum with fix outgoing angle

[64]:
```
## Spherical step potential which is U for r < R, and 0 outside
def spherical_step_potential(r, U, R):
    return -U if scipy.linalg.norm(r) < R else 0.0

def scattering_vs_E_born1(U=0.3,R=1.0):
    m=1.0
    xi=np.array([[1.0], [0.0]], dtype=complex)
    Nmc, Lmc = 10000, 1.0
    # U, R = 0.5, 1.0 # Potential parameters
    Emin, Emax, numE = 1.0, 5.0, 20
    Evec = np.linspace(Emin, Emax, numE)      # array of energies
    fvec = np.zeros(len(Evec), dtype=complex) # scattering amplitudes to␣
 ↪calculate
    for n in range(numE):
        k  = np.sqrt(Evec[n]**2-m**2)
        ki=np.array([0,0,k])
        kf= np.array([0,0,k])
        f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        fvec[n]=np.real_if_close(np.conj(f1order).T @ gamma_0 @ f1order).item()
    plt.plot(Evec, abs(fvec), 'o', label='First Born approx')
    plt.xlabel('E'); plt.ylabel('|f|^2'); plt.legend()
    plt.show()

scattering_vs_E_born1()
```

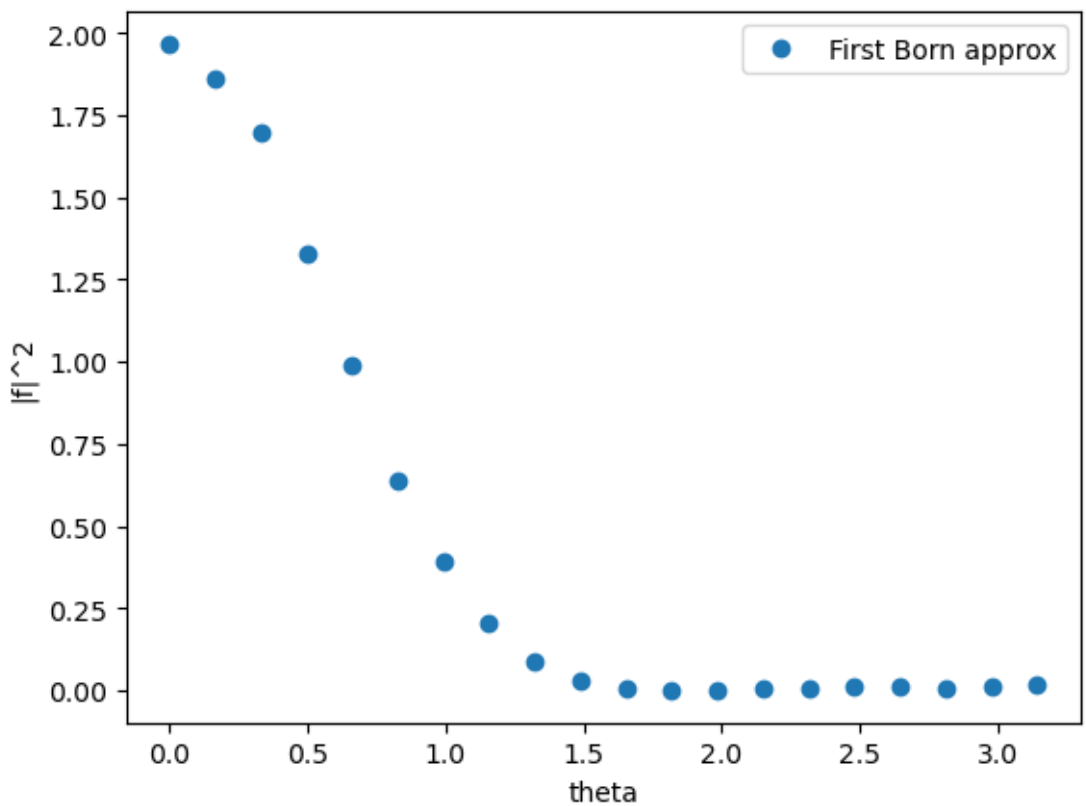


[65]:
```
scattering_vs_E_born1(U=-0.3,R=1.0)
```

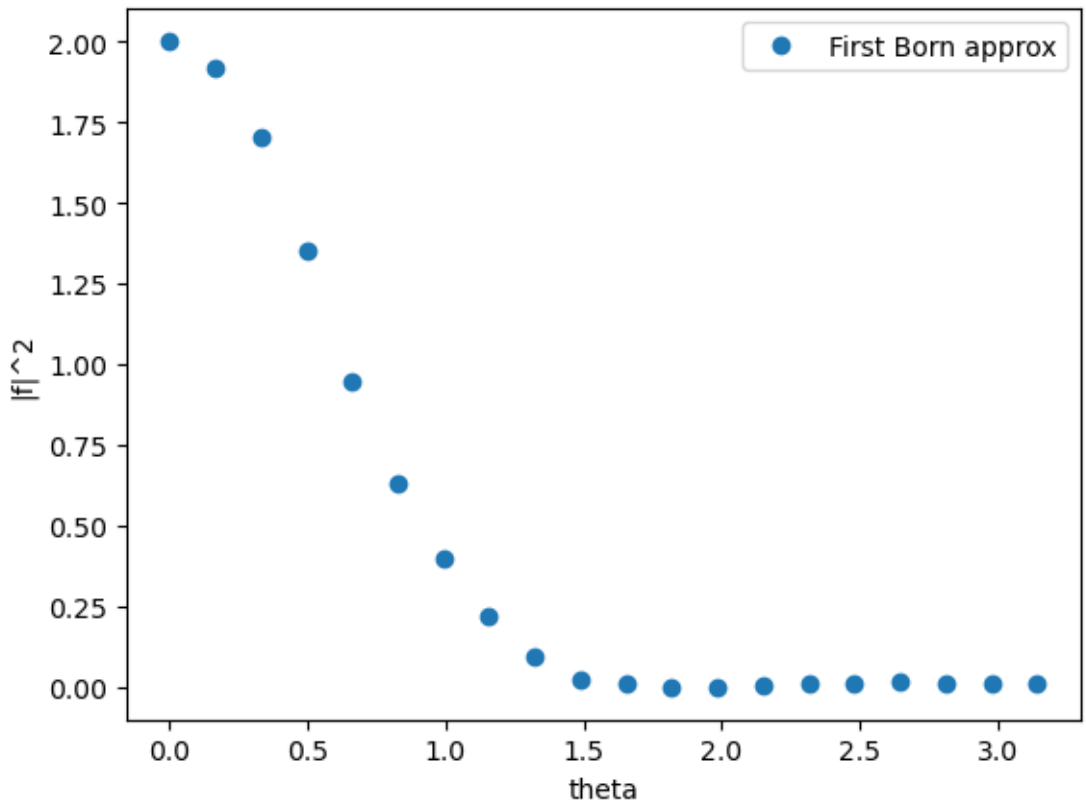


[66]:
```
def scattering_vs_angle_born1(U=0.5,R=1.0):
    m=1.0
    xi=np.array([[1.0], [0.0]], dtype=complex)
    Nmc, Lmc = 10000, 1.0
   # U, R = 0.5, 1.0 # Potential parameters
    E=3.0
    k  = np.sqrt(E**2-m**2)
    ki=np.array([0,0,k])
    thetamin, thetamax, numtheta = 0, np.pi, 20
    thetavec = np.linspace(thetamin, thetamax, numtheta)
    fvec = np.zeros(len(thetavec), dtype=complex) # scattering amplitudes to␣
↪calculate
    phi=0
    for n in range(numtheta):
        theta=thetavec[n]
        kf= k*np.array([np.sin(theta)*np.cos(phi),np.sin(theta)*np.sin(phi),np.
↪cos(theta)])
        f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        fvec[n]=np.real_if_close(np.conj(f1order).T @ gamma_0 @ f1order).item()
    plt.plot(thetavec, abs(fvec), 'o', label='First Born approx')
    plt.xlabel('theta'); plt.ylabel('|f|^2'); plt.legend()
```

```
    # phi=1
    # for n in range(numtheta):
    #     theta=thetavec[n]
    #     kf= k*np.array([np.sin(theta)*np.cos(phi),np.sin(theta)*np.
↪sin(phi),np.cos(theta)])
    #     f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
    #     fvec[n]=np.real_if_close(np.conj(f1order).T @ gamma_0 @ f1order).
↪item()
    # plt.plot(thetavec, abs(fvec), 'o', label='First Born approx')
    # plt.xlabel('theta'); plt.ylabel('|f|^2'); plt.legend()
    plt.show()
scattering_vs_angle_born1(U=0.5,R=1.0)
```



[14]:
```
scattering_vs_angle_born1(U=-0.5,R=1.0)
```



1.3 Second Born Approximation

For the second order, we have the following

$$f_2 = \int d^3r' d^3r'' \, \frac{(E + m\gamma^0 - \gamma^3\gamma^0 k) e^{i(k_i \cdot \mathbf{r}'' - k_f \cdot \mathbf{r}')}}{4\pi} V(\mathbf{r}') \, \frac{(E + m\gamma^0 - \gamma^3\gamma^0 k - i\gamma^3\gamma^0/R) e^{ikR}}{4\pi R} V(r'') u^s_{k_i}$$

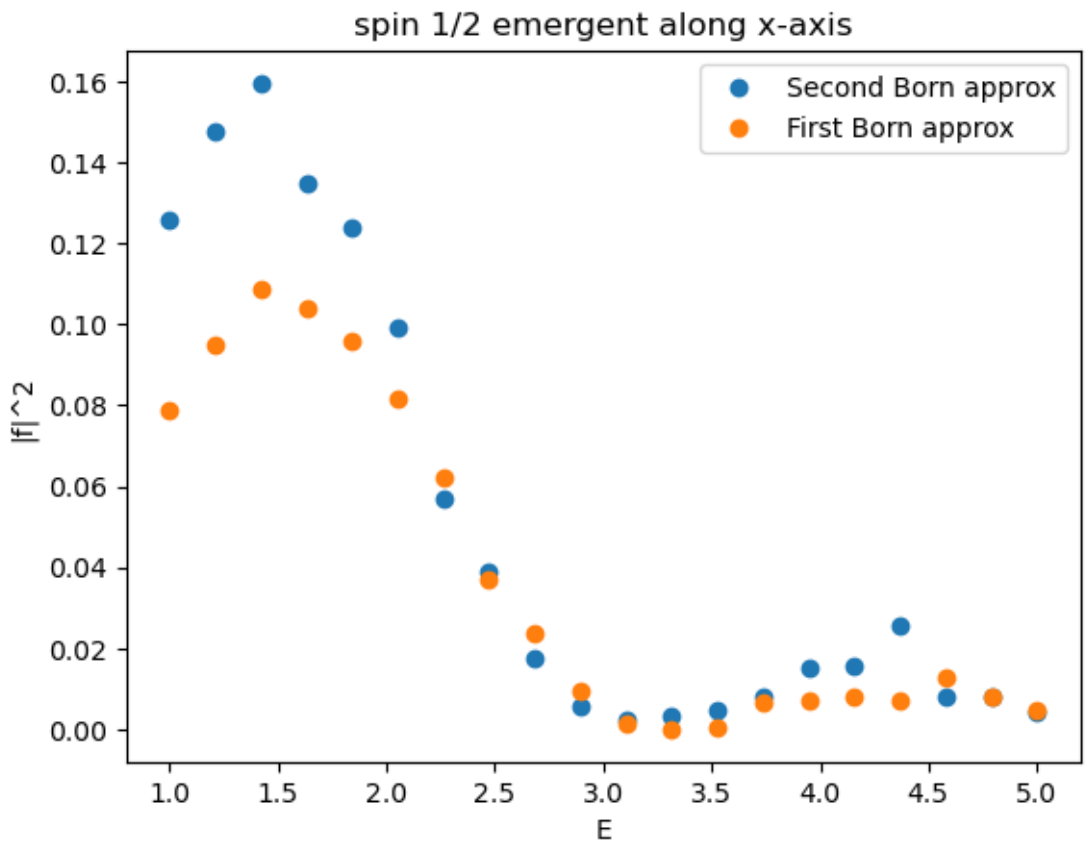
[7]:
```
## Function to estimate the first Born contribution to the scattering amplitude
 ↪f.
## The inputs are:
##
## Vfun   -- A function object.  The function should take an array [x, y, z]
##           and (optionally) other parameters. It should return V(x,y,z).
## ki, kf -- Initial and final wavevectors, which should have equal magnitudes.
## L      -- Size of sampling box.  Each coordinate is drawn from [-L, L].
## N      -- Number of Monte Carlo runs; default is 5000.
## Vargs  -- A tuple of additional parameters to pass Vfun.

def fborn2(Vfun, ki, kf, L, N, xi, m=1, Vargs=()):
    volume = (2*L)**6     # Spatial volume of integration
    f2 = np.zeros((4, 1), dtype=complex)            # Scattering amplitude
```

```
    for n in range(N):
        r1  = 2 * L * np.random.random(3) - L # Each coordinate is -L to L
        r2  = 2 * L * np.random.random(3) - L # Each coordinate is -L to L
        dkr = np.dot(ki, r1)-np.dot(kf, r2)
        k=np.sqrt(np.dot(ki, ki))
        R=np.sqrt(np.dot(r1-r2, r1-r2))
        dkR = np.dot(k, R)
        E=np.sqrt(np.dot(kf, kf)+m**2)
        p=np.concatenate(([E], ki))
        ui=u_spinor_weyl(p, xi)
        f2 +=  Vfun(r1, *Vargs)* Vfun(r2, *Vargs) * np.exp(1j *␣
↪dkr)*(E*I4+m*gamma_0-k*gamma_3@gamma_0) / (4 * np.pi)* np.exp(1j * dkR)@\
        (E*I4+m*gamma_0-k*gamma_3@gamma_0-1j*gamma_3@gamma_0/R)@ui /(4*np.pi*R)
    return volume * f2 / N
```

1.3.1 Energy Spectrum with different emergent angles

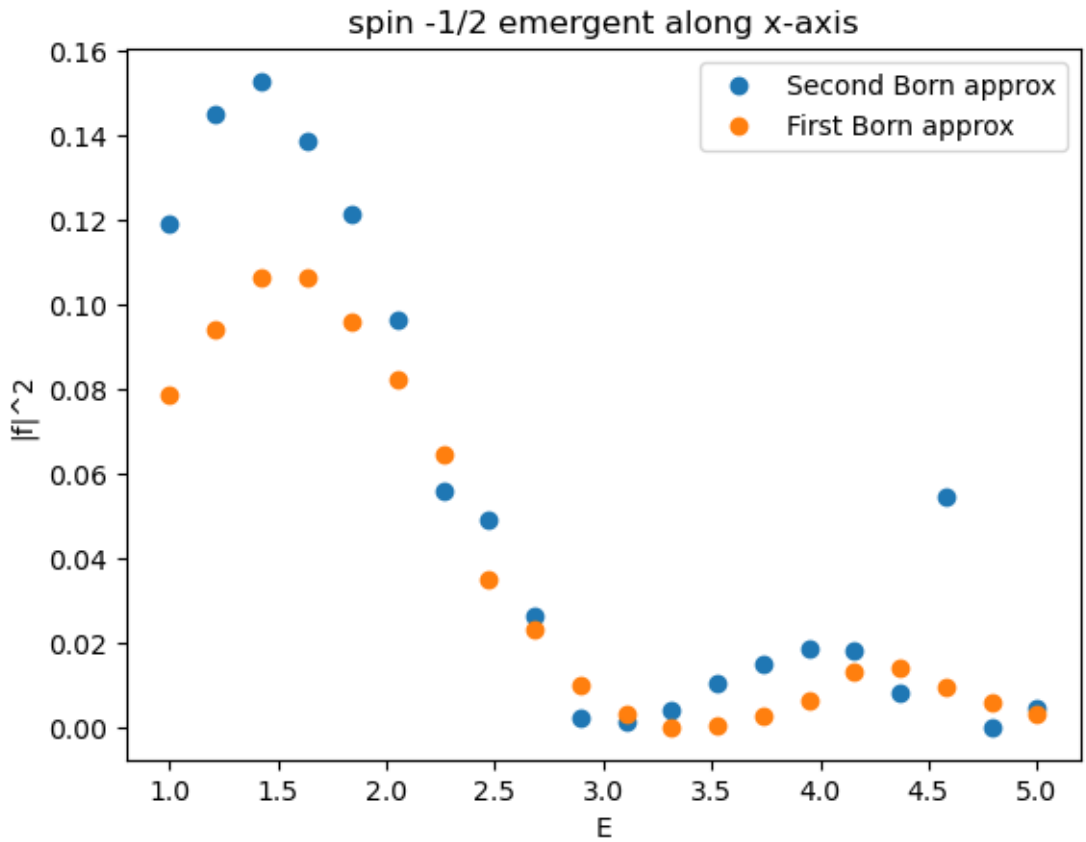
[160]:
```
def scattering_vs_E_born2(U=-0.3,R=1.0):
    m=1.0
    xi=np.array([[1.0], [0.0]], dtype=complex)
    Nmc, Lmc = 10000, 1.0
    # U, R = 0.5, 1.0 # Potential parameters
    Emin, Emax, numE = 1, 5.0, 20
    Evec = np.linspace(Emin, Emax, numE)      # array of energies
    fvec = np.zeros(len(Evec), dtype=complex) # scattering amplitudes to␣
↪calculate
    for n in range(numE):
        k  = np.sqrt(Evec[n]**2-m**2)
        ki=np.array([0,0,k])
        kf= np.array([k,0,0])
        f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        f2order=fborn2(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        ftot=f1order+f2order
        fvec[n]=np.real_if_close(np.conj(ftot).T @ gamma_0 @ ftot).item()
        # fvec[n]=np.real_if_close(np.conj(f2order).T @ gamma_0 @ f2order).
↪item()
    plt.plot(Evec, abs(fvec), 'o', label='Second Born approx')
    for n in range(numE):
        k  = np.sqrt(Evec[n]**2-m**2)
        ki=np.array([0,0,k])
        kf= np.array([k,0,0])
        f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        # f2order=fborn2(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        # ftot=f1order+f2order
        # fvec[n]=np.real_if_close(np.conj(ftot).T @ gamma_0 @ ftot).item()
        fvec[n]=np.real_if_close(np.conj(f1order).T @ gamma_0 @ f1order).item()
    plt.plot(Evec, abs(fvec), 'o', label='First Born approx')
```

```
    plt.title("spin 1/2 emergent along x-axis")
    plt.xlabel('E'); plt.ylabel('|f|^2'); plt.legend()
    plt.savefig('spin 1-2 emergent along x-axis', format='pdf')
    plt.show()

scattering_vs_E_born2()
```



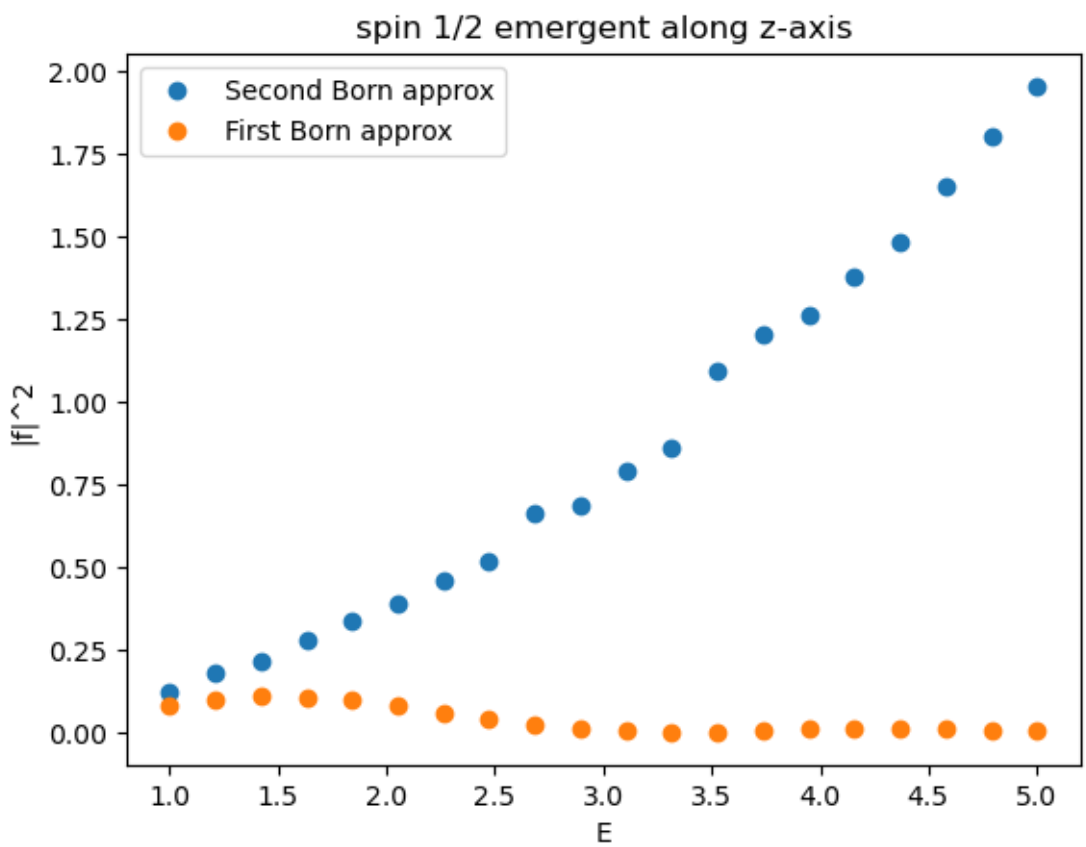
[161]:
```
def scattering_vs_E_born2(U=-0.3,R=1.0):
    m=1.0
    xi=np.array([[0.0], [1.0]], dtype=complex)
    Nmc, Lmc = 10000, 1.0
    # U, R = 0.5, 1.0 # Potential parameters
    Emin, Emax, numE = 1, 5.0, 20
    Evec = np.linspace(Emin, Emax, numE)      # array of energies
    fvec = np.zeros(len(Evec), dtype=complex) # scattering amplitudes to␣
 ↪calculate
    for n in range(numE):
        k  = np.sqrt(Evec[n]**2-m**2)
        ki=np.array([0,0,k])
```

```
        kf= np.array([k,0,0])
        f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        f2order=fborn2(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        ftot=f1order+f2order
        fvec[n]=np.real_if_close(np.conj(ftot).T @ gamma_0 @ ftot).item()
        # fvec[n]=np.real_if_close(np.conj(f2order).T @ gamma_0 @ f2order).
↪item()
    plt.plot(Evec, abs(fvec), 'o', label='Second Born approx')
    for n in range(numE):
        k  = np.sqrt(Evec[n]**2-m**2)
        ki=np.array([0,0,k])
        kf= np.array([k,0,0])
        f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        # f2order=fborn2(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        # ftot=f1order+f2order
        # fvec[n]=np.real_if_close(np.conj(ftot).T @ gamma_0 @ ftot).item()
        fvec[n]=np.real_if_close(np.conj(f1order).T @ gamma_0 @ f1order).item()
    plt.plot(Evec, abs(fvec), 'o', label='First Born approx')
    plt.title("spin -1/2 emergent along x-axis")
    plt.xlabel('E'); plt.ylabel('|f|^2'); plt.legend()
    plt.savefig('spin -1-2 emergent along x-axis', format='pdf')
    plt.show()

scattering_vs_E_born2()
```

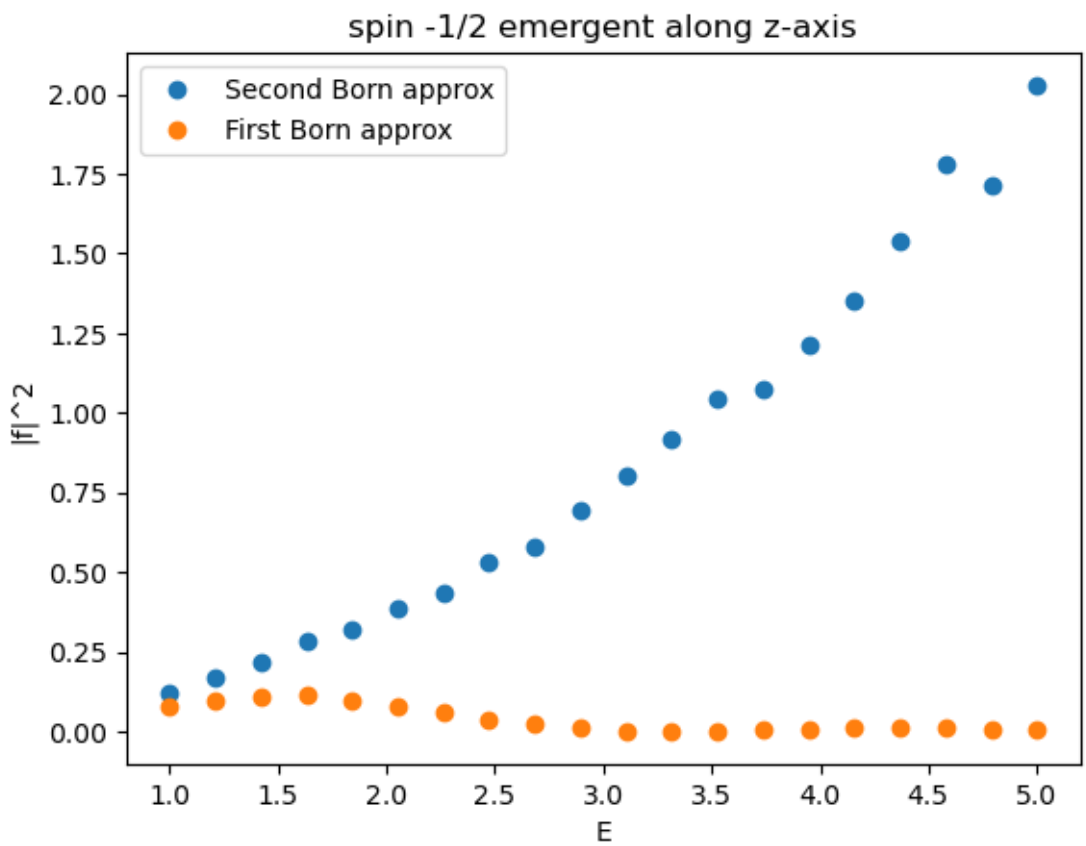


[162]:
```
def scattering_vs_E_born2(U=-0.3,R=1.0):
    m=1.0
    xi=np.array([[1.0], [0.0]], dtype=complex)
    Nmc, Lmc = 10000, 1.0
    # U, R = 0.5, 1.0 # Potential parameters
    Emin, Emax, numE = 1, 5.0, 20
    Evec = np.linspace(Emin, Emax, numE)      # array of energies
    fvec = np.zeros(len(Evec), dtype=complex) # scattering amplitudes to␣
↪calculate
    for n in range(numE):
        k  = np.sqrt(Evec[n]**2-m**2)
        ki=np.array([0,0,k])
        kf= np.array([0,0,k])
        f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        f2order=fborn2(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        ftot=f1order+f2order
        fvec[n]=np.real_if_close(np.conj(ftot).T @ gamma_0 @ ftot).item()
        # fvec[n]=np.real_if_close(np.conj(f2order).T @ gamma_0 @ f2order).
↪item()
```

```
    plt.plot(Evec, abs(fvec), 'o', label='Second Born approx')
    for n in range(numE):
        k  = np.sqrt(Evec[n]**2-m**2)
        ki=np.array([0,0,k])
        kf= np.array([k,0,0])
        f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        # f2order=fborn2(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        # ftot=f1order+f2order
        # fvec[n]=np.real_if_close(np.conj(ftot).T @ gamma_0 @ ftot).item()
        fvec[n]=np.real_if_close(np.conj(f1order).T @ gamma_0 @ f1order).item()
    plt.plot(Evec, abs(fvec), 'o', label='First Born approx')
    plt.title("spin 1/2 emergent along z-axis")
    plt.xlabel('E'); plt.ylabel('|f|^2'); plt.legend()
    plt.savefig('spin 1-2 emergent along z-axis', format='pdf')
    plt.show()

scattering_vs_E_born2()
```



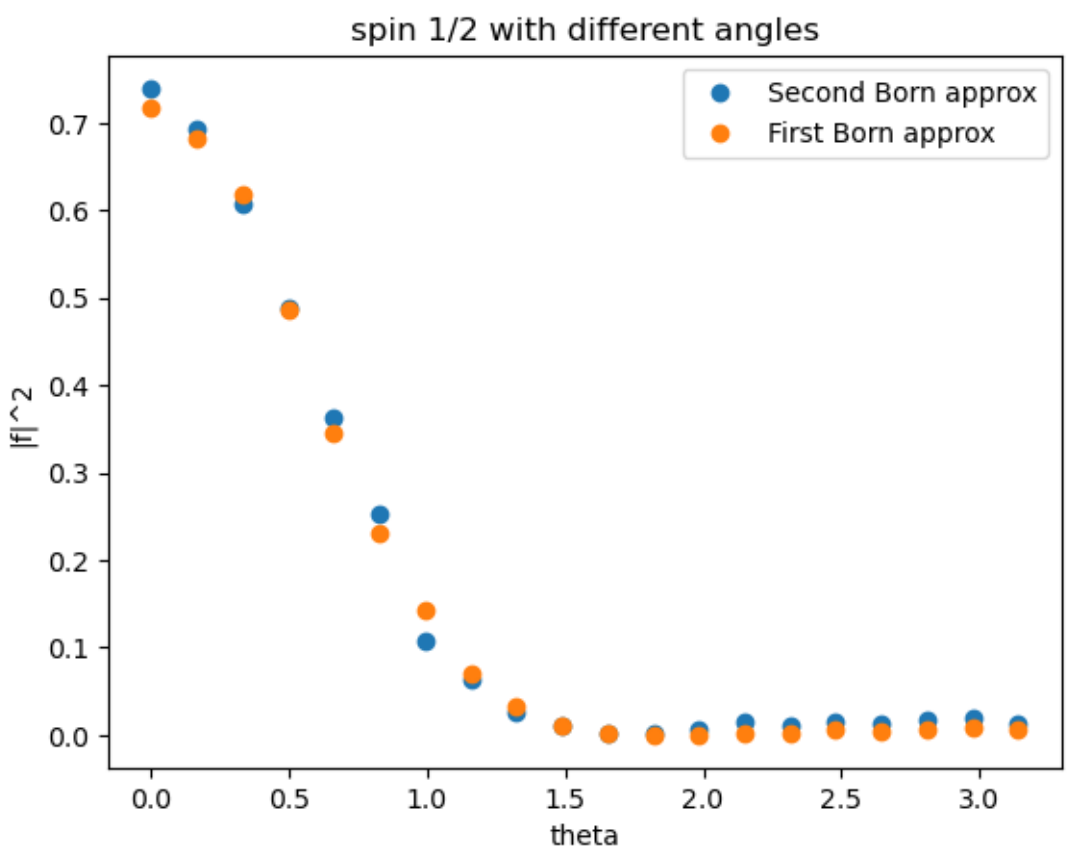
[163]:
```
def scattering_vs_E_born2(U=-0.3,R=1.0):
    m=1.0
    xi=np.array([[0.0], [1.0]], dtype=complex)
    Nmc, Lmc = 10000, 1.0
    # U, R = 0.5, 1.0 # Potential parameters
    Emin, Emax, numE = 1, 5.0, 20
    Evec = np.linspace(Emin, Emax, numE)      # array of energies
    fvec = np.zeros(len(Evec), dtype=complex) # scattering amplitudes to␣
 ↪calculate
    for n in range(numE):
        k  = np.sqrt(Evec[n]**2-m**2)
        ki=np.array([0,0,k])
        kf= np.array([0,0,k])
        f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        f2order=fborn2(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        ftot=f1order+f2order
        fvec[n]=np.real_if_close(np.conj(ftot).T @ gamma_0 @ ftot).item()
        # fvec[n]=np.real_if_close(np.conj(f2order).T @ gamma_0 @ f2order).
 ↪item()
    plt.plot(Evec, abs(fvec), 'o', label='Second Born approx')
    for n in range(numE):
        k  = np.sqrt(Evec[n]**2-m**2)
        ki=np.array([0,0,k])
        kf= np.array([k,0,0])
        f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        # f2order=fborn2(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        # ftot=f1order+f2order
        # fvec[n]=np.real_if_close(np.conj(ftot).T @ gamma_0 @ ftot).item()
        fvec[n]=np.real_if_close(np.conj(f1order).T @ gamma_0 @ f1order).item()
    plt.plot(Evec, abs(fvec), 'o', label='First Born approx')
    plt.title("spin -1/2 emergent along z-axis")
    plt.xlabel('E'); plt.ylabel('|f|^2'); plt.legend()
    plt.savefig('spin -1-2 emergent along z-axis', format='pdf')
    plt.show()

scattering_vs_E_born2()
```



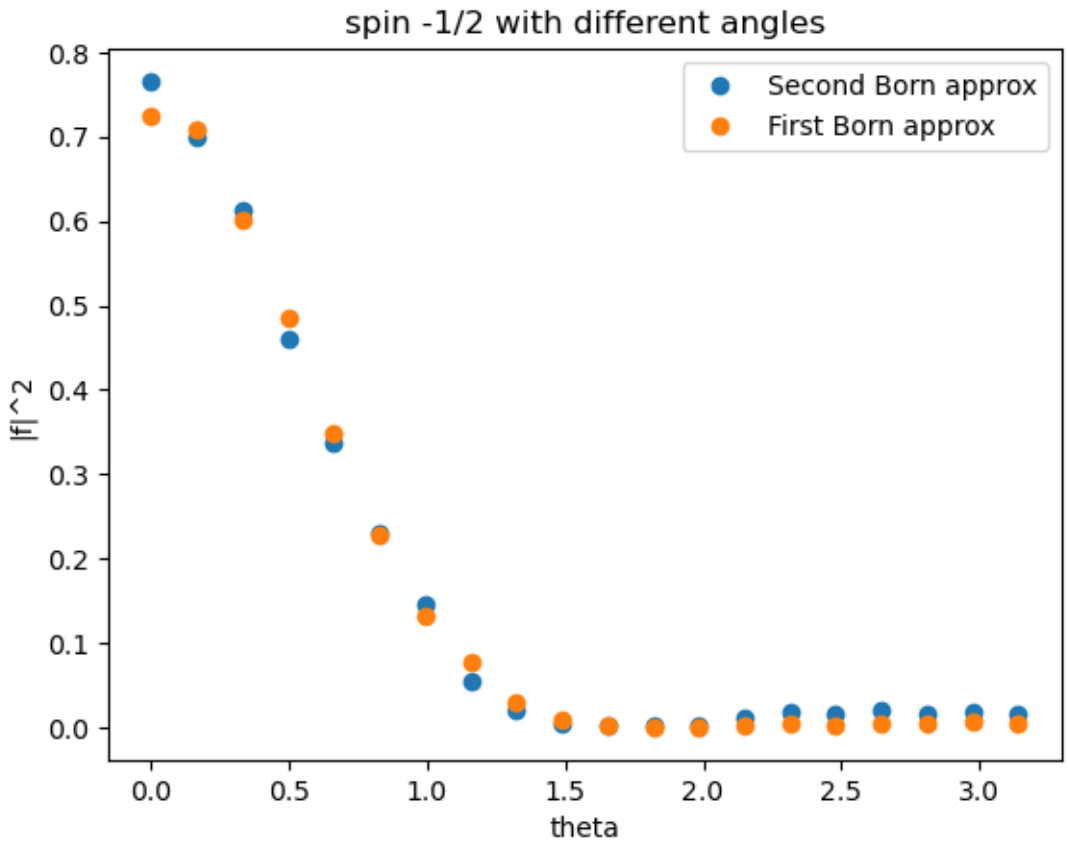
1.3.2 Scattering with different angles

[164]:
```
def scattering_vs_angle_born2(U=-0.3,R=1.0):
    m=1.0
    xi=np.array([[1.0], [0.0]], dtype=complex)
    Nmc, Lmc = 10000, 1.0
    #U, R = 0.5, 1.0 # Potential parameters
    E=3.0
    k  = np.sqrt(E**2-m**2)
    ki=np.array([0,0,k])
    thetamin, thetamax, numtheta = 0, np.pi, 20
    thetavec = np.linspace(thetamin, thetamax, numtheta)
    fvec = np.zeros(len(thetavec), dtype=complex) # scattering amplitudes to␣
↪calculate
    phi=1
    for n in range(numtheta):
        theta=thetavec[n]
        kf= k*np.array([np.sin(theta)*np.cos(phi),np.sin(theta)*np.sin(phi),np.
↪cos(theta)])
```

```
        f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        f2order=fborn2(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        ftot=f1order+f2order
        fvec[n]=np.real_if_close(np.conj(ftot).T @ gamma_0 @ ftot).item()
        # fvec[n]=np.real_if_close(np.conj(f2order).T @ gamma_0 @ f2order).
↪item()
    plt.plot(thetavec, abs(fvec), 'o', label='Second Born approx')
    plt.xlabel('theta'); plt.ylabel('|f|^2'); plt.legend()
    # phi=1
    for n in range(numtheta):
        theta=thetavec[n]
        kf= k*np.array([np.sin(theta)*np.cos(phi),np.sin(theta)*np.sin(phi),np.
↪cos(theta)])
        f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        fvec[n]=np.real_if_close(np.conj(f1order).T @ gamma_0 @ f1order).item()
    plt.plot(thetavec, abs(fvec), 'o', label='First Born approx')
    plt.xlabel('theta'); plt.ylabel('|f|^2'); plt.legend()
    plt.title("spin 1/2 with different angles")
    plt.savefig('spin 1-2 with different angles', format='pdf')
    plt.show()
scattering_vs_angle_born2(U=-0.3,R=1.0)
```



[165]:
```
def scattering_vs_angle_born2(U=-0.3,R=1.0):
    m=1.0
    xi=np.array([[0.0], [1.0]], dtype=complex)
    Nmc, Lmc = 10000, 1.0
    #U, R = 0.5, 1.0 # Potential parameters
    E=3.0
    k  = np.sqrt(E**2-m**2)
    ki=np.array([0,0,k])
    thetamin, thetamax, numtheta = 0, np.pi, 20
    thetavec = np.linspace(thetamin, thetamax, numtheta)
    fvec = np.zeros(len(thetavec), dtype=complex) # scattering amplitudes to␣
 ↪calculate
    phi=1
    for n in range(numtheta):
        theta=thetavec[n]
        kf= k*np.array([np.sin(theta)*np.cos(phi),np.sin(theta)*np.sin(phi),np.
 ↪cos(theta)])
        f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        f2order=fborn2(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        ftot=f1order+f2order
        fvec[n]=np.real_if_close(np.conj(ftot).T @ gamma_0 @ ftot).item()
        # fvec[n]=np.real_if_close(np.conj(f2order).T @ gamma_0 @ f2order).
 ↪item()
    plt.plot(thetavec, abs(fvec), 'o', label='Second Born approx')
    plt.xlabel('theta'); plt.ylabel('|f|^2'); plt.legend()
    # phi=1
    for n in range(numtheta):
        theta=thetavec[n]
        kf= k*np.array([np.sin(theta)*np.cos(phi),np.sin(theta)*np.sin(phi),np.
 ↪cos(theta)])
        f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        fvec[n]=np.real_if_close(np.conj(f1order).T @ gamma_0 @ f1order).item()
    plt.plot(thetavec, abs(fvec), 'o', label='First Born approx')
    plt.xlabel('theta'); plt.ylabel('|f|^2'); plt.legend()
    plt.title("spin -1/2 with different angles")
    plt.savefig('spin -1-2 with different angles', format='pdf')
    plt.show()
scattering_vs_angle_born2(U=-0.3,R=1.0)
```



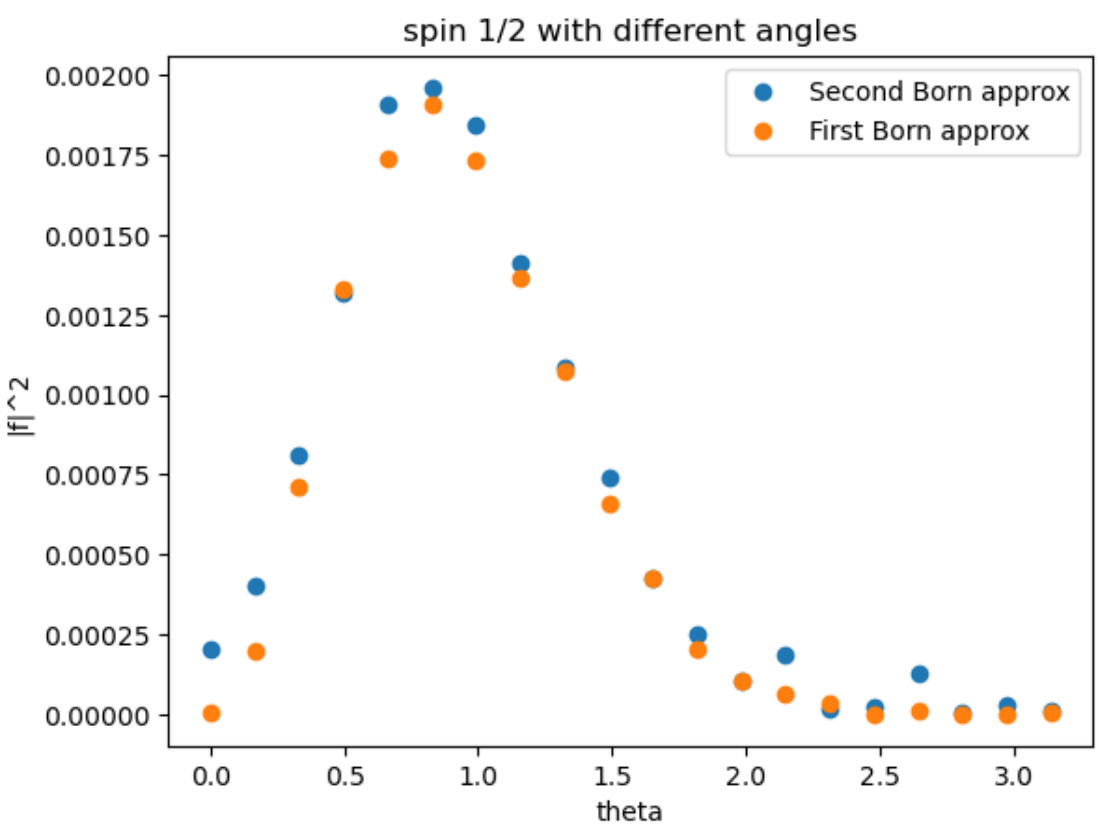
1.4 Adding Magnetic Potential

[15]:
```
def spherical_stern_gerlach_potential(r, U, R):
    x = r[0]
    if scipy.linalg.norm(r) < R:
        return -U * x * np.kron(np.identity(2), sigma_x)  #
    else:
        return np.zeros((4, 4))


def fborn1(Vfun, ki, kf, L, N, xi, m=1, Vargs=()):
    volume = (2*L)**3     # Spatial volume of integration
    f1 = np.zeros((4, 1), dtype=complex)            # Scattering amplitude
    k=np.sqrt(np.dot(ki,ki))
    for n in range(N):
        r1  = 2 * L * np.random.random(3) - L # Each coordinate is -L to L
        dkr = np.dot(ki - kf, r1)
        E=np.sqrt(np.dot(kf, kf)+m**2)
        p=np.concatenate(([E], ki))
        ui=u_spinor_weyl(p, xi)
```

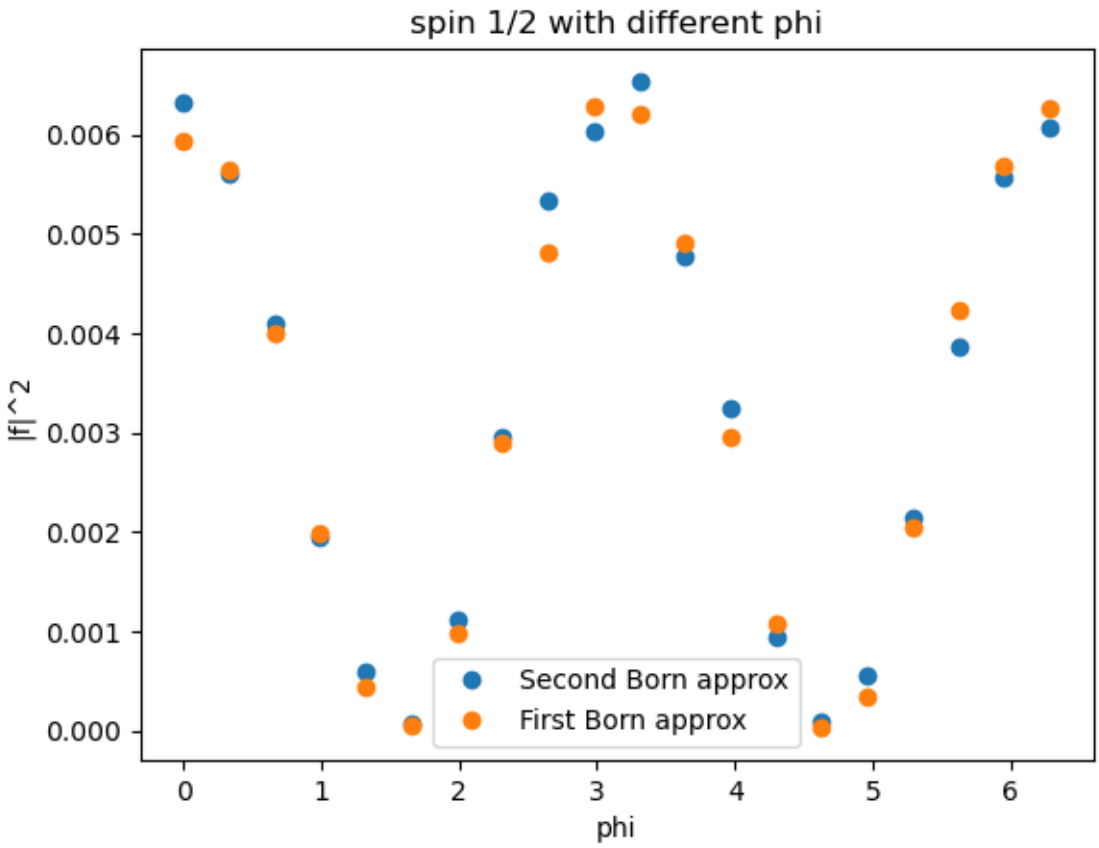
```
        f1 +=   np.exp(1j * dkr)*(E*I4+m*gamma_0-k*gamma_3 @ gamma_0)@Vfun(r1,␣
↪*Vargs) @ ui / (4 * np.pi)
    return volume * f1 / N


def fborn2(Vfun, ki, kf, L, N, xi, m=1, Vargs=()):
    volume = (2*L)**6    # Spatial volume of integration
    f2 = np.zeros((4, 1), dtype=complex)            # Scattering amplitude
    for n in range(N):
        r1  = 2 * L * np.random.random(3) - L # Each coordinate is -L to L
        r2  = 2 * L * np.random.random(3) - L # Each coordinate is -L to L
        dkr = np.dot(ki, r1)-np.dot(kf, r2)
        k=np.sqrt(np.dot(ki, ki))
        R=np.sqrt(np.dot(r1-r2, r1-r2))
        dkR = np.dot(k, R)
        E=np.sqrt(np.dot(kf, kf)+m**2)
        p=np.concatenate(([E], ki))
        ui=u_spinor_weyl(p, xi)
        f2 +=  np.exp(1j * dkr)*(E*I4+m*gamma_0-k*gamma_3@gamma_0) / (4 * np.
↪pi) @ Vfun(r1, *Vargs) * np.exp(1j * dkR)@\
        (E*I4+m*gamma_0-k*gamma_3@gamma_0-1j*gamma_3@gamma_0/R)@Vfun(r2,␣
↪*Vargs) @ui /(4*np.pi*R)
    return volume * f2 / N
```

[17]:
```
def scattering_vs_angle_born2(U=-0.3,R=1.0):
    m=1.0
    xi=np.array([[1.0], [0.0]], dtype=complex)
    Nmc, Lmc = 10000, 1.0
    #U, R = 0.5, 1.0 # Potential parameters
    E=3.0
    k  = np.sqrt(E**2-m**2)
    ki=np.array([0,0,k])
    thetamin, thetamax, numtheta = 0, np.pi, 20
    thetavec = np.linspace(thetamin, thetamax, numtheta)
    fvec = np.zeros(len(thetavec), dtype=complex) # scattering amplitudes to␣
↪calculate
    phi=1
    for n in range(numtheta):
        theta=thetavec[n]
        kf= k*np.array([np.sin(theta)*np.cos(phi),np.sin(theta)*np.sin(phi),np.
↪cos(theta)])
        ␣
↪f1order=fborn1(spherical_stern_gerlach_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        ␣
↪f2order=fborn2(spherical_stern_gerlach_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        ftot=f1order+f2order
```

```
        fvec[n]=np.real_if_close(np.conj(ftot).T @ gamma_0 @ ftot).item()
        # fvec[n]=np.real_if_close(np.conj(f2order).T @ gamma_0 @ f2order).
↪item()
    plt.plot(thetavec, abs(fvec), 'o', label='Second Born approx')
    plt.xlabel('theta'); plt.ylabel('|f|^2'); plt.legend()
    # phi=1
    for n in range(numtheta):
        theta=thetavec[n]
        kf= k*np.array([np.sin(theta)*np.cos(phi),np.sin(theta)*np.sin(phi),np.
↪cos(theta)])
        ␣
↪f1order=fborn1(spherical_stern_gerlach_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        fvec[n]=np.real_if_close(np.conj(f1order).T @ gamma_0 @ f1order).item()
    plt.plot(thetavec, abs(fvec), 'o', label='First Born approx')
    plt.xlabel('theta'); plt.ylabel('|f|^2'); plt.legend()
    plt.title("spin 1/2 with different angles")
    plt.savefig('SG_ spin_z 1-2 with different theta', format='pdf')
    plt.show()
scattering_vs_angle_born2(U=-0.3,R=1.0)
```



[35]:
```
def scattering_vs_angle_born2(U=-0.3,R=1.0):
    m=1.0
    xi=np.array([[1.0], [0.0]], dtype=complex)
    Nmc, Lmc = 10000, 1.0
    #U, R = 0.5, 1.0 # Potential parameters
    E=3.0
    k  = np.sqrt(E**2-m**2)
    ki=np.array([0,0,k])
    phimin, phimax, numphi = 0, 2*np.pi, 20
    phivec = np.linspace(phimin, phimax, numphi)
    fvec = np.zeros(len(phivec), dtype=complex) # scattering amplitudes to␣
↪calculate
    theta=0.9
    for n in range(numphi):
        phi=phivec[n]
        kf= k*np.array([np.sin(theta)*np.cos(phi),np.sin(theta)*np.sin(phi),np.
↪cos(theta)])
        ␣
↪f1order=fborn1(spherical_stern_gerlach_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        ␣
↪f2order=fborn2(spherical_stern_gerlach_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        ftot=f1order+f2order
        fvec[n]=np.real_if_close(np.conj(ftot).T @ gamma_0 @ ftot).item()
        # fvec[n]=np.real_if_close(np.conj(f2order).T @ gamma_0 @ f2order).
↪item()
    plt.plot(phivec, abs(fvec), 'o', label='Second Born approx')
    plt.xlabel('phi'); plt.ylabel('|f|^2'); plt.legend()
    # phi=1
    for n in range(numphi):
        phi=phivec[n]
        kf= k*np.array([np.sin(theta)*np.cos(phi),np.sin(theta)*np.sin(phi),np.
↪cos(theta)])
        ␣
↪f1order=fborn1(spherical_stern_gerlach_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        fvec[n]=np.real_if_close(np.conj(f1order).T @ gamma_0 @ f1order).item()
    plt.plot(phivec, abs(fvec), 'o', label='First Born approx')
    plt.xlabel('phi'); plt.ylabel('|f|^2'); plt.legend()
    plt.title("spin 1/2 with different phi")
    plt.savefig('SG_ spin_z 1-2 with different phi', format='pdf')
    plt.show()
scattering_vs_angle_born2(U=-0.3,R=1.0)
```

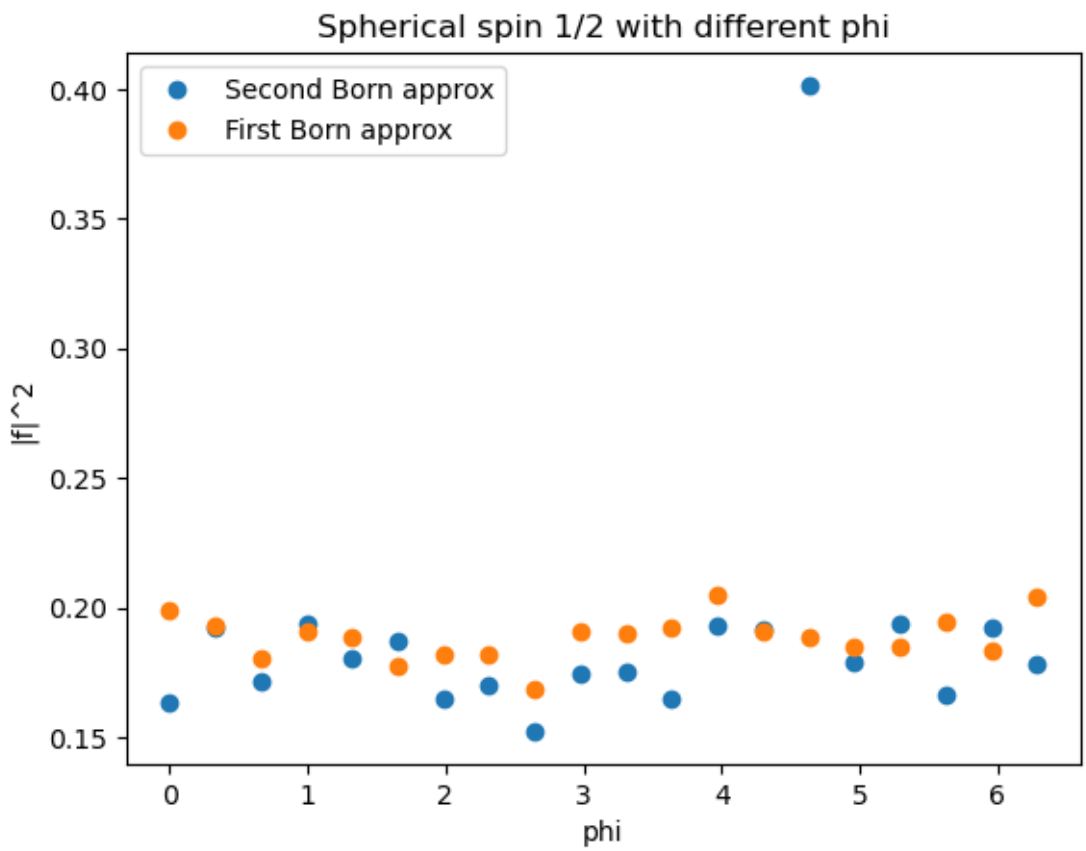


[83]:
```
# def scattering_vs_angle_born2(U=-0.3,R=1.0):
#     m=1.0
#     xi=np.array([[1.0], [-1.0]], dtype=complex)/np.sqrt(2)
#     Nmc, Lmc = 10000, 1.0
#     #U, R = 0.5, 1.0 # Potential parameters
#     E=3.0
#     k  = np.sqrt(E**2-m**2)
#     ki=np.array([0,0,k])
#     phimin, phimax, numphi = 0, 2*np.pi, 20
#     phivec = np.linspace(phimin, phimax, numphi)
#     intensity_plus = np.zeros(len(phivec), dtype=complex) # scattering␣
 ↪amplitudes to calculate
#     intensity_minus = np.zeros(len(phivec), dtype=complex) # scattering␣
 ↪amplitudes to calculate
#     theta=0.8

#     Sigma_x = np.kron(np.eye(2), sigma_x)
#     P_xplus = 0.5 * (Sigma_x + np.eye(4))
#     P_xminus = 0.5 * (np.eye(4) - Sigma_x)
```

```
#     for n in range(numphi):
#         phi=phivec[n]
#         kf= k*np.array([np.sin(theta)*np.cos(phi),np.sin(theta)*np.
 ↪sin(phi),np.cos(theta)])
#         ␣
 ↪f1order=fborn1(spherical_stern_gerlach_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
#         ␣
 ↪f2order=fborn2(spherical_stern_gerlach_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
#         ftot=f1order+f2order
#         proj_plus = P_xplus @ ftot
#         proj_minus = P_xminus @ ftot
#         intensity_plus[n] = np.real((proj_plus.conj().T @ gamma_0 @␣
 ↪proj_plus).item())
#         intensity_minus[n] = np.real((proj_minus.conj().T @ gamma_0 @␣
 ↪proj_minus).item())
#     plt.plot(phivec, abs(intensity_plus), 'o', label='plus')
#     plt.plot(phivec, abs(intensity_minus), 'o', label='minus')
#     plt.xlabel('phi'); plt.ylabel('|f|^2'); plt.legend()
#     # phi=1
#     # for n in range(numphi):
#     #     phi=phivec[n]
#     #     kf= k*np.array([np.sin(theta)*np.cos(phi),np.sin(theta)*np.
 ↪sin(phi),np.cos(theta)])
#     #    ␣
 ↪f1order=fborn1(spherical_stern_gerlach_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
#     #     proj_plus = P_xplus @ f1order
#     #     proj_minus = P_xminus @ f1order
#     #     intensity_plus[n] = np.real((proj_plus.conj().T @ gamma_0 @␣
 ↪proj_plus).item())
#     #     intensity_minus[n] = np.real((proj_minus.conj().T @ gamma_0 @␣
 ↪proj_minus).item())
#     # plt.plot(phivec, abs(fvec), 'o', label='First Born approx')
#     # plt.xlabel('phi'); plt.ylabel('|f|^2'); plt.legend()
#     plt.title("spin 1/2 and spin-1/2 with different phi")
#     plt.savefig('SG_ spin_z pm1-2 with different phi', format='pdf')
#     plt.show()
# scattering_vs_angle_born2(U=0.5,R=1.0)
```

[41]:
```
def spherical_step_potential(r, U, R):
    return -U *np.eye(4) if scipy.linalg.norm(r) < R else 0.0*np.eye(4)
def scattering_vs_phi_born2(U=-0.3,R=1.0):
    m=1.0
    xi=np.array([[1.0], [0.0]], dtype=complex)
    Nmc, Lmc = 10000, 1.0
```

```
    #U, R = 0.5, 1.0 # Potential parameters
    E=3.0
    k  = np.sqrt(E**2-m**2)
    ki=np.array([0,0,k])
    phimin, phimax, numphi = 0, 2*np.pi, 20
    phivec = np.linspace(phimin, phimax, numphi)
    fvec = np.zeros(len(phivec), dtype=complex) # scattering amplitudes to␣
↪calculate
    theta=0.9
    for n in range(numphi):
        phi=phivec[n]
        kf= k*np.array([np.sin(theta)*np.cos(phi),np.sin(theta)*np.sin(phi),np.
↪cos(theta)])
        f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        f2order=fborn2(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        ftot=f1order+f2order
        fvec[n]=np.real_if_close(np.conj(ftot).T @ gamma_0 @ ftot).item()
        # fvec[n]=np.real_if_close(np.conj(f2order).T @ gamma_0 @ f2order).
↪item()
    plt.plot(phivec, abs(fvec), 'o', label='Second Born approx')
    plt.xlabel('phi'); plt.ylabel('|f|^2'); plt.legend()
    # phi=1
    for n in range(numphi):
        phi=phivec[n]
        kf= k*np.array([np.sin(theta)*np.cos(phi),np.sin(theta)*np.sin(phi),np.
↪cos(theta)])
        f1order=fborn1(spherical_step_potential,ki,kf,Lmc,Nmc,xi,m,(U,R))
        fvec[n]=np.real_if_close(np.conj(f1order).T @ gamma_0 @ f1order).item()
    plt.plot(phivec, abs(fvec), 'o', label='First Born approx')
    plt.xlabel('phi'); plt.ylabel('|f|^2'); plt.legend()
    plt.title("Spherical spin 1/2 with different phi")
    plt.savefig('Spherical_ spin_z 1-2 with different phi', format='pdf')
    plt.show()
scattering_vs_phi_born2(U=-0.3,R=1.0)
```


Spherical spin 1/2 with different phi
Second Born approx
First Born approx
0.40
0.35
0.30
0.25
0.20
0.15
|f|^2
0
1
2
3
4
5
6
phi

[ ]: